

Kapitel 17. Mera om klasser

Innehållsförteckning

[Dynamiskt allokera objekt](#)[Friends](#)[Polymorfism](#)[Standardvärden för funktioner och metoder](#)[Statiska metoder och medlemmar](#)[Pekaren `this`](#)

Detta kapitel behandlar olika aspekter på hur klasser fungerar i C++. Många små ämnen har här kombinerats ihop till ett kapitel.

Dynamiskt allokera objekt

Förutom normala primitiva datatyper, sammansatta datatyper och vektorer kan man även allokera objekt dynamiskt. Dynamiskt allokerade objekt skiljer sig väldigt lite från *automatiska* objekt, d.v.s. objekt som skapas automatiskt av kompilatorn i någon viss funktion eller metod. Fördelarna med att dynamiskt allokera objekt när de behövs är desamma som för andra typer av minne:

- mindre slöseri med minne.
- programmet är mera dynamiskt i olika situationer.

Allmänt ser en allokering av en klass ut på följande sätt:

```
klasstyp * variabel1 = new klasstyp;  
klasstyp * variabel1 = new klasstyp (parametrar);
```

Vi antar att vi har en klass dom heter `Coordinate` definierad (se [Kapitel 14](#)). Vi kan allokera en dylik genom följande anrop:

```
// använd ursprungsvärden  
Coordinate * C1 = new Coordinate;  
  
// använd egna värden  
Coordinate * C2 = new Coordinate ( 1, 2 );
```

Vilken metod som används beror på klassen ifråga och vilka olika konstruktörer denna har. Vår klass `Coordinate` har både en tom konstruktor och en konstruktor som tar två parametrar. Allokerade objekt måste även förstöras för att inte läcka minne. Det görs som normalt med operatören `delete`. För att deallokera ovan allokerade objekt använder vi:

```
delete C1;  
delete C2;
```

Man bör minnas att det endas sätt att referera till ett allokerat objekt är via en pekare som pekar till objektet. Finns det ingen pekare längre som pekar på objektet är det förlorat för all framtid och kan

inte igen användas. Vi har då läckt minne.

För att allokeras en vektor av objekt används samma syntax som för primitiva datatyper:

```
Coordinate * Vector = new Coordinate [42];
```

Man adresserar sedan denna på samma sätt som normala vektorer, d.v.s. via []. En vektor av objekt raderas även på samma sätt som en normal vektor:

```
delete [] Vector;
```

Då objekt allokeras med `new` och förstörs med `delete` exekveras deras konstruktörer och destruktörer på normalt sätt.

Exempel på dynamisk minneshantering

Ett exempel på dynamisk minneshantering följer i programmet nedan. Det använder sig även av *interna klasser* (se [avsnittet *Klasser i klasser i Kapitel 14*](#)) och illustrerar hur dessa kan användas. Programmet implementerar ett mycket enkelt binärt träd som lagrar heltal. Man kan sätta in tal i trädet samt skriva ut trädet. Trädet använder sig av rekursiva hjälpfunktioner för att skriva ut trädet samt lägga in element.

```
#include <iostream>
#include <stdlib.h>
#include <time.h>

class Tree {
public:
    // konstruktor
    Tree () : m_Root (0) {};
    ~Tree ();

    // lägg till ett tal
    void add (const int Value);

    // skriv ut trädet
    void print () { print ( m_Root ); }

private:
    // intern klass för att hålla data om en nod
    class Node {
    public:
        Node (const int Value) : m_Value(Value), m_Left(0), m_Right(0) {};
        ~Node ();

        // värdet för noden
        int m_Value;

        // vänster och höger subträd
        Node * m_Left;
        Node * m_Right;
    };

    // rekursiv metod för att lägga till ett värde och skriva ut
    void add (const int Value, Node * Root);
    void print (Node * Root);

    // roten för trädet
```

```
Node * m_Root;
};

Tree::~Tree () {
    // har vi en rot?
    if ( m_Root )
        delete m_Root;
}

void Tree::add (const int Value) {
    // har vi en rot redan?
    if ( ! m_Root )
        // nej, så skapa en rot och gå iväg
        m_Root = new Node ( Value );

    else
        // lägg till värdet till roten
        add (Value, m_Root);
}

void Tree::add (const int Value, Node * Root) {
    // kolla värdet för denna nod
    if ( Value < Root->m_Value ) {
        // har vi en gren till vänster?
        if ( Root->m_Left ) {
            // jep, rekursera vidare
            add (Value, Root->m_Left);
        }
        else {
            // nej, så skapa en nod till vänster
            Root->m_Left = new Node (Value);
        }
    }

    // kolla värdet för denna nod
    else if ( Value > Root->m_Value ) {
        // har vi en gren till höger?
        if ( Root->m_Right ) {
            // jep, rekursera vidare
            add (Value, Root->m_Right);
        }
        else {
            // nej, så skapa en nod till vänster
            Root->m_Right = new Node (Value);
        }
    }
}

void Tree::print (Node * Root) {
    // har vi ett värde i noden?
    if ( ! Root )
        // nej, gå bort
        return;

    // skriv ut vänster subträd
    print ( Root->m_Left );
    cout << Root->m_Value << " ";

    // skriv ut höger subträd
    print ( Root->m_Right );
}

Tree::Node::~Node () {
    // har vi ett vänster subträd?
```

```
    if ( m_Left )
        delete m_Left;

    // har vi ett höger subträd?
    if ( m_Right )
        delete m_Right;
}

int main () {
    Tree T;

    // initiera slumpalen
    srand ( time (0) );

    // sätt in 20 slumpal i trädet
    for ( int Index = 0; Index < 20; Index++ ) {
        T.add ( (int)((float)rand () / (float)RAND_MAX * 100.0) );
    }

    // skriv ut trädet
    T.print ();
}
```

Logiken i programmet är relativt enkel och kräver inga närmare beskrivningar. Notera dock hur destruktorn för den interna klassen `Node` implementerats. Eftersom den är intern för klassen `Tree` räcker det inte med enbart:

```
Node::~~Node () {
    ...
}
```

utan man måste även berätta att det är en metod för en klass som tillhör `Tree` genom att även skriva `Tree::` enligt:

```
Tree::Node::~~Node () {
    ...
}
```

[Förutgående](#)
Multipel ärvning

[Hem](#)

[Nästa](#)
Friends